

MCP vs Skills: Two Ways to Extend an LLM's Context

Understand when to wire an LLM to live external data with MCP, and when to teach it a repeatable procedure with a skill.

For developers building AI agents or coding assistants who need to decide how to extend an LLM beyond its training data · 27 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

[Watch the source video](#)[Read the full lesson](#)[Read the condensed version](#)

What matters

- ✓ An LLM answers well only when its context window contains the right information, not just a well-phrased question — this is the difference between prompt engineering and context engineering.
- ✓ MCP (Model Context Protocol) standardizes how an LLM-based application calls external services: it turns a service's API into an LLM-readable set of tools, and it handles scoped, authenticated access to that service.
- ✓ Without MCP, giving a model raw API documentation and a token and hoping it calls the API correctly is fragile and unauditable.
- ✓ MCP does not make an LLM's behavior repeatable — LLMs are non-deterministic, so even with the same data source, output formatting and process can vary run to run.
- ✓ A skill is a packaged, reusable set of instructions (typically a markdown file plus metadata, optionally with scripts and resources) that teaches the model to perform a task the same way every time.
- ✓ Skills are auto-loaded into the context window only when relevant, so they don't bloat every request with instructions the current task doesn't need.
- ✓ Choose MCP when the agent needs controlled, permissioned access to real-time external data. Choose a skill when you need a lightweight, repeatable way of doing something the model doesn't already know how to do consistently.
- ✓ MCP and skills are complementary, not competing: an agent can use MCP to fetch live CRM data and a skill to guarantee that data is always formatted and processed the same way.

What it covers

- 01 From prompt engineering to context engineering** 0:00
An LLM is a pattern-matching system trained on huge amounts of text; the quality of its answer depends on what extra information you give it beyond the question itself.
- 02 MCP: a standard interface between an LLM and external services** 2:55
The Model Context Protocol standardizes how an AI application calls external data sources and services, so the model doesn't need raw API docs and a bare token to get live data.
- 03 What MCP doesn't solve: repeatable, non-deterministic behavior** 3:39
LLMs are non-deterministic, so even with reliable access to live data via MCP, the model can format or process that data differently each time unless it's told a fixed procedure to follow.
- 04 Skills: packaging a repeatable procedure the model can auto-load** 5:08

A skill is a reusable bundle — a markdown file with metadata, optionally packaged with scripts and resources in a folder — that teaches the model to do a specific task the same way every time, loaded into context only when needed.

05 Choosing MCP, skills, or both 5:53

Reach for MCP when the agent needs controlled, permissioned access to real-time external data; reach for a skill when you need a lightweight, repeatable way to perform a task the model doesn't already do consistently.

Glossary

Context window — The span of text an LLM can see at once when generating a response, including the prompt, any supplied data, and prior conversation.

Context engineering — Deciding what information — data, formatting rules, tool output — needs to be present in the model's context window for it to answer correctly, beyond just the wording of the prompt.

MCP server — A server that implements MCP for a specific service, exposing a set of typed tools to the AI application and translating tool calls into that service's real API requests.

Skill — A reusable, packaged set of instructions — typically a markdown file with metadata, optionally bundled with scripts and resources — that teaches an LLM to perform a specific task the same way every time, loaded into context only when relevant.

Auto-loading (of a skill) — The behavior where an AI application matches the current task against each available skill's description and pulls only the matching skill's instructions into the context window.

Prompt engineering — Crafting the wording, role, and task of a request to an LLM to shape its response.

MCP (Model Context Protocol) — A standardized protocol that lets an AI application call external services and data sources through a common, LLM-readable interface, handling both request translation and authentication.

Scoped token — An access credential limited to exactly the permissions (e.g. read-only, or write access to one resource type) that a given task needs, rather than a broad, all-access credential.

Non-deterministic — Producing different outputs on different runs even given the same input, which is a property of how LLMs generate text and why unenforced instructions don't guarantee repeatable results.

JSON-RPC — A lightweight remote-procedure-call protocol encoded in JSON, used by MCP to structure requests like tools/list and tools/call between client and server.

Explanations are AI-generated. Check anything you intend to rely on.